Field Hours

# CSCE 312 Computer Organization

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 50 minutes

## Bits and two's complement

A bit is 0 or 1. Eight bits are a byte. An unsigned byte holds 0 through 255. Two's complement represents negatives by flipping the bits and adding one. The high bit is the sign. The range of an 8-bit two's complement number is -128 through 127. Adding one to 127 overflows to -128. That wrap is the bug, not a mystery.

Hexadecimal writes four bits as one character. 0xFF is 255 unsigned and -1 in eight-bit two's complement. Say the width before you say the value.

The same bits, two readings.

Unsigned

11111111 is 255

00000001 is 1

Two's complement

11111111 is -1

10000000 is -128

### Negate 5 in four bits

1. 5 is 0101. Flip to 1010. Add one: 1011.

2. That is -5. Check: 0101 + 1011 = 10000, and the carry off the top leaves 0000. Negatives add to zero.

Problem 1. An 8-bit counter sits at 255 and you add 1, unsigned. What is stored?

Answer. 0. The sum is 256, which needs a ninth bit. The stored byte wraps. If the type is signed, 255 is not a stored value; the bit pattern 11111111 is already -1, and adding 1 yields 0.

Problem 2. Why does two's complement use flip-and-add-one instead of a sign bit alone?

Answer. The same add circuit works for negative numbers. A separate sign bit needs a different subtraction path. Two's complement makes negation the flip-plus-one, and addition does the rest.

Lecture 2 · 50 minutes

## Gates and an adder

AND is 1 only if both inputs are 1. OR is 1 if either is 1. NOT flips a bit. XOR is 1 if the inputs differ. Those

four are enough to add. A one-bit full adder takes two bits and a carry-in, and produces a sum bit and a carry-out. The sum bit is the XOR of the three inputs. The carry-out is on when at least two inputs are 1.

A ripple adder chains the carry of one bit into the next. The last carry waits on every earlier bit. That delay is why a wide add is not instant, and it is the fact CSCE 614 will pipeline around.

One bit of an adder.

Two bits and a carry in -> Sum bit out -> Carry to the next bit

### Add 3 and 1 in two bits

1. 3 is 11, 1 is 01. Low bits 1 and 1, carry in 0. Sum bit 0, carry 1.

2. High bits 1 and 0, plus carry 1. Sum bit 0, carry 1.

3. Result bits 00 with a carry out, which is 4. Two bits cannot hold 4. The carry out is the overflow signal.

Problem 1. XOR of 1 and 1 is 0. AND of 1 and 1 is 1. Which one is the sum bit of those two inputs with no carry, and which is the carry?

Answer. XOR is the sum bit, 0. AND is the carry, 1. That is 1 + 1 = 2, written as carry 1 and sum 0.

Problem 2. A 32-bit ripple adder's last bit depends on the first carry chain. Why does that matter?

Answer. The high bit cannot be trusted until the carry has rippled through all 32 stages. A clock that ticks sooner samples a wrong sum. Faster adders reduce that chain. The graduate course will hide the remaining latency in a pipeline.

Lecture 3 · 50 minutes

## An instruction and a register

A processor has a small set of registers and a memory. An instruction names an operation and the registers it uses. A load copies memory into a register. A store copies a register into memory. An add reads two registers and writes one. The program counter is a register that holds the address of the next instruction.

Assembly is those instructions written so a person can read them. A compiler emits them. You should be able to read a three-instruction sequence: load, add, store. That is a variable increment.

x = x + 1, in three instructions.

1. Load x into a register

2. Add one

3. Store the register back to x

### A two-register add

1. R1 holds 4. R2 holds 7. ADD R3, R1, R2 writes 11 into R3.

2. R1 and R2 are unchanged. The destination is a third name.

3. If the instruction overwrote R1, the old 4 would be gone. Read the destination field before you assume the sources survive.

Problem 1. Why can the processor not add two memory locations in the instruction above?

Answer. This instruction set adds registers. Memory is reached by load and store. A machine that added memory to memory would be a different instruction set. Know which one you are reading.

Problem 2. The program counter holds 100 and the instruction is 4 bytes. After a normal instruction, what is the program counter?

Answer. 104. A branch writes a different address instead. Sequential code is just adding the instruction length.

Lecture 4 · 50 minutes

# A single-cycle datapath

In one clock, a simple processor fetches an instruction, reads registers, computes, and writes the result. The clock must be long enough for the slowest of those paths, which is usually a load: fetch, address add, memory read, register write. Every instruction pays that length, even an add that does not touch memory.

That waste is why the next course pipelines. This course stops when you can point at the path an add takes and the path a load takes, and say which is longer.

One clock, one instruction.

Fetch -> Read registers -> ALU -> Memory, if a load or store -> Write the register

## A load versus an add

1. The add uses fetch, register read, the ALU, and a register write. It does not use data memory.

2. The load uses all of that and a memory read. The clock is set by the load.

3. The add finishes early and waits. A single-cycle machine does not start the next instruction sooner. The pipeline course is the repair.

Problem 1. You shorten the ALU and the clock does not speed up. What is the clock waiting on?

Answer. The longer path, typically instruction fetch plus a data-memory read on a load. Speeding a piece that is not on the longest path does not change the clock.

Problem 2. Why is 'one instruction per clock' not the same as 'fast'?

Answer. The clock may be slow, because it must cover the longest instruction. Many short clocks in a pipeline can finish more instructions per second than one long clock, even if each instruction takes several of the short clocks.

Lecture 5 · 50 minutes

# Memory the processor sees

Registers are the fastest storage and there are few of them. Main memory is large and slow. A cache sits between

them and holds the lines you used recently. A hit is a load that finds its line in the cache. A miss goes to memory and pays the long delay.

You do not need the graduate cache geometry yet. You need the consequence: a loop that touches the same small array is mostly hits. A loop that jumps through a huge array is mostly misses. The algorithm's order is a memory behavior.

Where a load looks.

Register

Cache

Main memory

### A loop over four words, then over a million

1. Four words fit in a tiny cache. After the first pass, the second pass hits.

2. A million words, touched once, miss if the cache cannot hold them. The processor waits on memory.

3. The add instruction is the same. The time is not.

Problem 1. Two versions of a sum walk the same array, one by rows and one by a stride of 1024. Why might the stride be slower even though both add the same number of times?

Answer. The stride misses more often. Consecutive words share a cache line. A stride of 1024 may touch a new line every add. The arithmetic count hid the memory cost.

Problem 2. Name the three stores in order of size and speed, registers first.

Answer. Registers, small and fast. Cache, larger and slower. Main memory, largest and slowest. A disk is slower still and is not what a load instruction touches directly.

Lecture 6 · 50 minutes

## The call

A function call saves the return address, passes arguments, and jumps. The convention says which registers hold arguments and which the callee must preserve. The stack holds the return address and any local values that do not fit in registers. A return reads the saved address and jumps back.

If you overwrite the saved return address, the return goes where the new value says. That is the mechanical fact the security prerequisite will call a buffer overflow. Here, just see that the address is data sitting next to the locals.

A stack frame.

Arguments

Return address

Saved registers

Local variables

## Call, then return

1. The caller writes the return address, the instruction after the call, and jumps to the function.

2. The function writes a local at the next stack slot.

3. Return loads the saved address and jumps there. The local is abandoned. If the local overflowed onto the return address, the jump is no longer the caller's next instruction.

Problem 1. A callee uses a register the convention says the caller still needs. Who saves it?

Answer. Whoever the convention assigns. If it is caller-saved, the caller stores it before the call and reloads it after. If it is callee-saved, the function stores it on entry and restores it before return. Breaking the convention corrupts the caller without a type error.

Problem 2. Why is the return address on the stack a security boundary, not only a calling detail?

Answer. It is writable memory next to data the function fills from the outside. An unchecked copy can replace it. The return then transfers control to an address the caller did not choose. CSCE 465 starts from this layout.